

AI-Based Detection of Cyber Threats in Unidirectional IP Traffic

Problem Statement 26145 - National Technical Research Organisation (NTRO)
Complete end-to-end reference: problem, theory, design, build, demo and defence

Problem Statement ID	26145
Organisation / Dept	National Technical Research Organisation (NTRO)
Category / Theme	Software - Blockchain & Cybersecurity
Deliverable	Working prototype (source repo) + model documentation + live dashboard
Hard constraints	Read-only ingest - no decryption - streaming - stated throughput - standard alert schema
Reference codename	Enclave Threat Console

Everything in this document is designed so that a person new to the project can read it front to back and be able to build, run, explain and defend the solution. Sample addresses use the documentation ranges reserved by RFC 5737 (192.0.2.0/24, 198.51.100.0/24, 203.0.113.0/24) and RFC 6598 shared space (100.64.0.0/10).

0. How to use this document

This is the single reference for the whole project. Sections 1 to 7 explain the problem and the theory; sections 8 to 13 are the design; section 14 is the built prototype; sections 15 to 24 cover data, novelty, limits, deployment, compliance, the team plan, judge questions, the pitch, checklists and references.

If you are...	Read first	Then
The presenter	Summary, 2, 8, 13, 22	21 (question bank), 16 (novelty)
The backend / ML developer	6, 7, 9, 10, 11, 12, 14	15 (data), 17 (limits)
The frontend developer	13, 11, 14	22 (what the demo must show)
The documentation owner	2, 10, 17, 19	24 (references)
Reviewing the whole idea	Read it in order	-

The one-sentence version. Traffic is copied one way into a sealed monitoring room; our system turns that one-way stream into real-time, scored, explainable alerts for six threat classes, using only metadata - never decrypting, never probing, never blocking.

The 60-second summary

- **Setting.** Critical-infrastructure operators mirror their gateway/peering links into a monitoring enclave through a passive TAP or a hardware data diode. Traffic can go in; nothing can come back out.
- **Problem.** Classic security tools assume a return path (probing, blocking, live lookups) or payload visibility (decryption). Behind a diode both assumptions fail, so operators end up with raw captures and slow, manual analysis.
- **Goal.** An AI/ML pipeline that ingests the one-way stream and detects, classifies and scores six threat classes in near real time from passive metadata alone, with evidence and a dashboard.
- **Our answer.** One streaming pipeline: ingest (pcap / NetFlow / IPFIX / sFlow) -> flow and metadata extraction -> six detectors (statistics + machine learning) -> fusion (calibrated confidence, asset-weighted severity, de-duplication, campaign correlation) -> standardised alerts (hash-chained for chain of custody) -> live dashboard.
- **Why it wins.** Constraint-first design that is tested, not just claimed; every alert explains itself; it degrades gracefully to flow-only inputs; it is fully offline and open; and it reports measured throughput, latency and false-alarm rates instead of only accuracy.

1. Glossary - the vocabulary you must be fluent in

Judges probe vocabulary. Every team member should be able to define these without hesitation.

Term	Meaning in this project
Data diode	Hardware that physically allows data to travel in only one direction (a transmitter on one side, a receiver on the other, no return laser). Not a firewall rule - a physical property.
TAP	Test Access Point: a passive optical/copper splitter that copies link traffic to a monitor port without affecting the link.
SPAN / port mirroring	A switch feature that copies traffic from ports to a monitor port. Software-configured, can drop packets under load.
Packet broker	A device that aggregates, filters and de-duplicates traffic from many TAPs before the sensors.
Monitoring enclave	The isolated environment (rack/room/VLAN) behind the diode where our system runs. No route back to production.
Flow	One conversation, identified by the 5-tuple (src IP, src port, dst IP, dst port, protocol) plus counters: bytes, packets, duration, TCP flags.
NetFlow / IPFIX / sFlow	Router-exported flow summaries. NetFlow v5/v9 and IPFIX are per-flow records; sFlow sends 1-in-N sampled packet headers plus counters.
Community ID	A standard hash of a flow's 5-tuple (Corelight spec) so Zeek, Suricata and our pipeline name the same flow identically.
Active / idle timeout	Flow metering rules: export a long-running flow every N seconds (active) and close a silent flow after M seconds (idle).
JA3 / JA3S	MD5 fingerprints of the TLS ClientHello / ServerHello field ordering. Identifies the client or server software without decryption.
JA4	Successor to JA3, resistant to TLS extension shuffling (Chrome randomises extension order since 2023). Sorted, human-readable, and the client variant is BSD-licensed.
SNI	Server Name Indication: the hostname a TLS client asks for, visible in cleartext in most handshakes.
SPLT	Sequence of Packet Lengths and Times: the sizes and timing of the first N packets - the 'rhythm' of an encrypted conversation.
DGA	Domain Generation Algorithm: malware invents pseudo-random domain names daily to find its controller.
NXDOMAIN	DNS answer meaning 'this name does not exist'. DGA hosts generate many of them.
DNS tunnelling	Smuggling data inside DNS queries/answers (iodine, dnscat2): long encoded subdomains and TXT/NULL records.
C2 beaconing	An infected host checking in with its controller on a schedule; detectable by regular inter-arrival times.
Entropy (Shannon)	Bits of randomness in a distribution. High source-IP entropy means the sources are spread out - typical of spoofed floods.
PCR	Producer-Consumer Ratio = (bytes out - bytes in) / (bytes out + bytes in). +1 is pure upload, -1 pure download.
TRW	Threshold Random Walk (Jung et al., 2004): sequential hypothesis test that declares a scanner after a few failed first-contact attempts.
EWMA	Exponentially Weighted Moving Average: a cheap rolling baseline that forgets old behaviour gradually.
Calibration	Making a model's 0.9 actually mean 'right about 90% of the time' (isotonic or Platt scaling).
SHAP	A method that attributes a model's decision to individual features - the basis of our 'top factors' evidence.
OCSF / ECS	Open Cybersecurity Schema Framework / Elastic Common Schema: standard shapes for security events. Our alert schema follows the same spirit.
MITRE ATT&CK	A public catalogue of attacker techniques (e.g. T1046 Network Service Discovery). Each alert cites its technique IDs.
Chain of custody	The documented, tamper-evident trail proving evidence was not altered since capture.
Watermark (event time)	How far event time has advanced in a stream; used to decide when a time window can be closed.

Term	Meaning in this project
Backpressure	Slowing a producer when the consumer is behind. Files can be slowed; a live feed cannot, so it drops and counts.

2. The problem statement, decoded

2.1 What the PS says, in plain words

- Operators watch gateway and peering links by copying traffic one way into a monitoring enclave, using passive mirroring or a hardware diode.
- This is deliberate: it removes the risk of the monitoring system becoming a pivot into the core network, and it keeps a clean chain of custody for forensics.
- The cost: any intelligence layer in the enclave must work purely from passively observed data - packet captures, exported flow records and derived metadata.
- It may never probe, never complete a handshake with the source, and never push a mitigation back.
- The objective is an AI/ML pipeline that ingests a one-directional stream of simulated IP data and detects, classifies and scores threats in near real time, outputting labelled alerts, confidence scores and supporting evidence on a visualisation dashboard.

2.2 The six threat classes

#	Class	What the PS asks for	Passive signal we use
a	Volumetric / protocol DDoS	SYN floods, UDP reflection/amplification, spoofed-source floods from flow-level rate and source-IP entropy statistics	New-flow rate vs baseline, source-IP entropy, SYN-without-handshake ratio, response/request byte ratio, TTL spread
b	Botnet C2 beaconing	Periodicity and inter-arrival analysis on flows repeating toward a small set of destinations	Inter-arrival mean and coefficient of variation, session-size consistency, repetition count, destination prevalence
c	DGA domains and DNS tunnelling	Entropy/n-gram analysis of query names plus query-length and record-type anomalies	Character entropy, rare-bigram ratio, digit ratio, NXDOMAIN rate; subdomain length, unique-subdomain count, TXT/NULL share
d	Malware in encrypted sessions	TLS/QUIC metadata only (JA3/JA3S or JA4, packet-size and timing sequences), no decryption	JA3/JA4 fingerprint, local rarity, missing SNI, offline blocklist match, SPLT rhythm
e	Reconnaissance and port scanning	Fan-out from one source across many ports or hosts	Distinct targets, failed first-contact ratio, TRW score, vertical/horizontal/block shape
f	Data exfiltration	Asymmetric flow-volume anomalies and unusual outbound-to-inbound byte ratios	Bytes out per window vs host baseline, out/in ratio, PCR, first-contact destination, off-hours

2.3 The five architectural constraints (the grading rubric)

Constraint	What it forbids / requires	How we satisfy it	How we prove it
a. Read-only ingest	No return path, no live query to the source, no inline block	Listen-only sources; no active modules; a process-level egress guard raises on any outbound connect/sendto	Automated test: an outbound connection attempt from inside the pipeline raises EgressAttemptError; dashboard shows blocked attempts
b. No payload decryption	TLS/QUIC analysed from metadata only	Only the cleartext handshake, packet sizes and timing are parsed; QUIC Initial packets are deliberately left sealed even though their keys are public	Code review: no key material, no cipher code; the TLS parser stops after the ClientHello
c. Streaming, not batch	Incremental processing with bounded latency	Event-time windows, flow active timeout of 5 s, per-second detector flush, alerts emitted mid-flow	Latency measured per alert (ingest to emit) and reported as p50/p95 on the dashboard
d. Defined throughput target	State and demonstrate the rate tested	Target: 20,000 flows/s sustained on a 4-core laptop; benchmark harness replays at fixed rates	bench/ results table: rate, duration, p50/p95 latency, dropped records

Constraint	What it forbids / requires	How we satisfy it	How we prove it
e. Standardised alert schema	Structured records with timestamp, flow id, threat class, confidence, evidence	Pydantic model exported as JSON Schema; Community ID as the flow id; evidence, top factors, MITRE IDs and custody hashes included	GET /api/schema/alert returns the schema; every alert validates against it

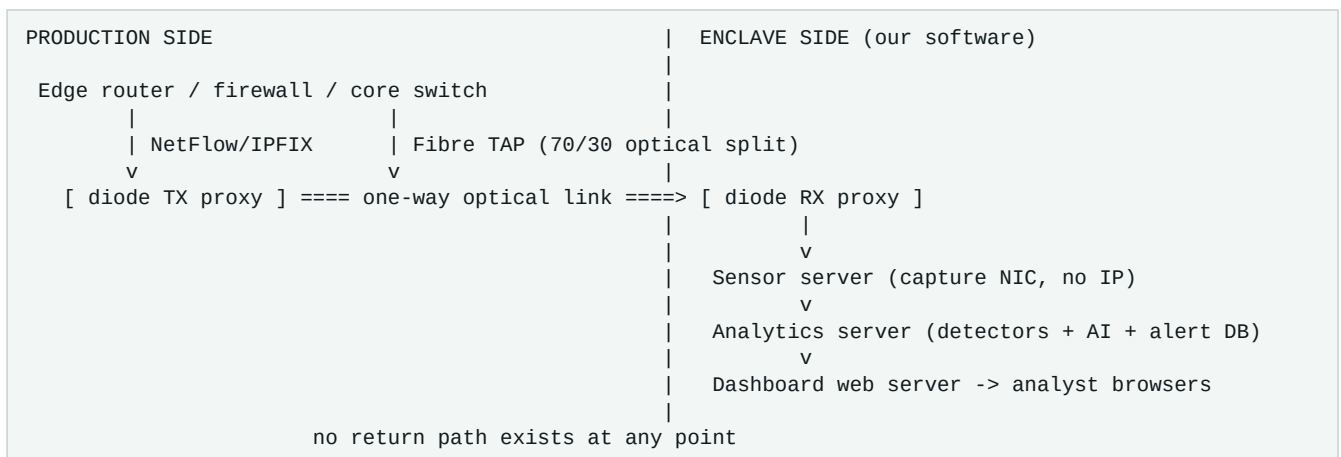
3. The environment: network, diode, enclave

3.1 What network is being watched

Zone	What lives there	Typical traffic
IT network	Staff workstations, email, web, ERP/finance	HTTPS, DNS, SMB, email, VPN
OT network	SCADA servers, HMIs, PLCs, RTUs, historians	Modbus/TCP (502), DNP3 (20000), IEC 60870-5-104 (2404), IEC 61850
DMZ	Public web, mail and DNS servers	HTTPS, DNS, SMTP
Gateway / peering links	Edge routers to ISPs and partner networks	Everything crossing the boundary; 1/10/40/100 Gbps

The PS targets the gateway and peering links. A useful second placement is the IT/OT boundary, where a Modbus sweep from an engineering laptop is far more alarming than the same sweep in IT.

3.2 The hardware chain, in order



Element	Role	Notes for the viva
Fibre TAP	Copies both directions of the link	Passive optics; cannot be hacked; needs a brief outage to install
SPAN / mirror port	Software alternative to a TAP	Free, but drops packets under load and depends on switch configuration
Packet broker	Aggregates and filters many TAPs	Optional; also de-duplicates and load-balances across sensors
Diode TX proxy	Terminates protocols on the production side and sends one way	Needed because TCP requires acknowledgements the diode cannot carry
One-way optical link	Transmitter one side, receiver the other	No laser on the enclave side means reverse transmission is physically impossible
Diode RX proxy	Rebuilds the stream inside the enclave	Uses forward error correction and redundancy because nothing can be re-sent
Sensor server	Runs the flow meter and metadata parsers	Capture interface is promiscuous and has no IP address
Analytics server	Runs the stream bus, detectors, models, alert store	CPU is sufficient; no GPU required for our models
Dashboard + analysts	Presentation and triage	Enclave LAN only; no internet access

3.3 Where our software plugs in

We install nothing on the production network and nothing on the diode itself. Our code starts at the output of the diode's receive side. The three hand-off methods a diode may offer are all supported by our ingest layer:

Hand-off	What the diode emits	What we do
Packet re-emit	Copied packets out an Ethernet port	Capture on a NIC with no IP address
UDP forward	NetFlow / IPFIX / sFlow records to an enclave address	Listen on the collector port (2055 / 4739 / 6343)
File drop	Small pcap chunks written to a folder or share every few seconds	Watch the folder and process each chunk on arrival (still streaming)

If a packet is lost inside the diode, nobody can ask for it again. The pipeline therefore tolerates gaps, counts dropped records, and never assumes complete coverage. This honesty is itself a differentiator - most academic pipelines assume a perfect capture.

4. What is the issue

4.1 Why the design exists

- A compromised monitoring or analytics system is a classic pivot into the core network. A diode removes that entire attack class physically.
- Evidence collected through a one-way path cannot be modified from the inside, which preserves the chain of custody for forensic and legal use.
- For OT/ICS sites, the safety case often depends on there being no possible write path to the control network.

4.2 What that costs the defender

A normal security tool can...	Behind the diode we cannot...
Probe or scan a suspicious host	Send any packet at all
Complete a TLS handshake to fingerprint a server (JARM)	Complete any handshake; only passive fingerprints (JA3/JA4/JA3S) are allowed
Decrypt HTTPS via an interception proxy	Decrypt anything - it also requires being inline
Look up WHOIS / reputation / geo APIs live	Query anything; intelligence must be imported offline
Block an attacker (firewall rule, TCP reset, BGP blackhole)	Take any action; humans act on the production side
Ask an endpoint agent what happened	See anything beyond the wire

4.3 The four technical difficulties

1. **Encryption.** The great majority of web and malware traffic is TLS or QUIC, so payload signatures are blind. Detection must come from behaviour and handshake metadata.
2. **Volume.** Gateway links generate tens of thousands of flows per second; detection must be incremental and memory-bounded, never a full-history join.
3. **Real time.** An end-of-run report is useless during a flood. Windows must close on event time with a bounded delay.
4. **Evasive adversaries.** Beacons add jitter, DGAs rotate daily, exfiltration goes slow and low, and TLS fingerprints can be copied from browsers.

5. What is done today, and why it falls short

Approach	Examples	Strength	Why it is not enough here
Signature IDS	Snort, Suricata rules	Precise on known payload patterns	Blind to encrypted payloads; needs a signature to exist; nothing for volumetric or behavioural threats
Passive network monitor	Zeek	Excellent structured logs (conn, dns, ssl, files)	Produces logs, not scored alerts; detection logic must still be written; no calibrated confidence
Beacon / DNS hunting	RITA	Good beacon and long-connection scoring	Works on periodically imported logs; covers a subset of the classes; no unified alerting
Flow / DDoS tools	FastNetMon, ntopng, nfdump	Fast volumetric alarms at line rate	Mostly static thresholds; mitigation features assume a return path (BGP flowspec, blackhole)
Commercial NDR	Darktrace, Vectra, ExtraHop, Corelight, Cisco Secure Network Analytics	Mature ML, encrypted-traffic analytics	Closed and costly; cloud or vendor-telemetry dependencies; automated response features conflict with a diode
SIEM	Splunk, ELK, Wazuh	Correlation and retention across sources	Query/rule based and usually delayed; no network-behaviour models out of the box
Academic ML-IDS	Papers on CIC-IDS2017/2018, UNSW-NB15	Published feature sets and high reported accuracy	Offline CSV experiments, random splits that leak, dataset artefacts, no streaming deployment, no explanations
Fingerprint blocklists	abuse.ch JA3 lists	Cheap detection of known malware stacks	Only catches known families; JA3 weakened by extension randomisation; needs offline updates

The gap in one line: there is no open, explainable, streaming pipeline that is designed from the ground up for a strictly one-way feed and covers all six classes with evidence, calibrated confidence and measured performance.

6. What input we actually receive

6.1 Three forms of input

Form	What it contains	Produced by	Detectors enabled
Raw packets (pcap/pcapng or live mirror)	Every packet: timestamp, headers, and payload bytes we deliberately ignore	TAP / SPAN via the diode	All six
Flow records (NetFlow v5/v9, IPFIX, sFlow)	Per-conversation summaries: 5-tuple, bytes, packets, start/end, TCP flags	Routers and switches, or softflowd from a pcap	DDoS, beaconing, scanning, exfiltration (4 of 6)
Derived metadata (Zeek/Suricata JSON, CICFlowMeter CSV)	Pre-extracted conn/dns/ssl logs or ~80 computed features per flow	Sensor tooling; public datasets	Depends on fields present; used mainly for model training

6.2 What the data looks like

A packet trace, as text

```
10:14:02.118392 IP 10.30.2.41.51522 > 203.0.113.66.443: Flags [S], seq 1829301, win 64240, length 0
10:14:02.160114 IP 203.0.113.66.443 > 10.30.2.41.51522: Flags [S.], ack 1829302, win 65160, length 0
10:14:02.160530 IP 10.30.2.41.51522 > 203.0.113.66.443: Flags [P.], length 517 <- TLS ClientHello (cleartext)
10:14:02.203871 IP 203.0.113.66.443 > 10.30.2.41.51522: Flags [P.], length 1410 <- encrypted, ignored
10:14:03.001220 IP 10.30.2.41.40211 > 10.10.0.8.53: UDP, A? kq7vzxrwpi3h.test <- DNS question (cleartext)
```

A normalised flow record (what our ingest produces)

```
{
  "flow_id": "1:LQU9qZLK+B5F3KDmev6m5PMibrg=",
  "src_ip": "10.30.2.41",
  "src_port": 51522,
  "dst_ip": "203.0.113.66",
  "dst_port": 443,
  "proto": 6,
  "first_ts": 1789034042.118,
  "last_ts": 1789034043.9,
  "bytes_fwd": 1180,
  "bytes_bwd": 1420,
  "pkts_fwd": 9,
  "pkts_bwd": 8,
  "syn_seen": true,
  "synack_seen": true,
  "rst_seen": false,
  "export_index": 0,
  "is_final": true,
  "min_ttl": 64,
  "max_ttl": 64,
  "splt": [517, -1410, -620, 93, -37],
  "mode": "pcap"
}
```

DNS and TLS metadata events

```
{
  "kind": "dns",
  "client_ip": "10.30.2.41",
  "resolver_ip": "10.10.0.8",
  "query": "kq7vzxrwpi3h.test",
  "qtype": 1,
  "is_response": false
},
{
  "kind": "dns",
  "client_ip": "10.30.2.41",
  "resolver_ip": "10.10.0.8",
  "query": "kq7vzxrwpi3h.test",
  "qtype": 1,
  "is_response": true,
  "rcode": 3
  # 3 = NXDOMAIN
},
{
  "kind": "tls",
  "client_ip": "10.30.5.23",
  "server_ip": "198.51.100.77",
  "server_port": 443,
  "version": "12",
  "sni": null,
  "ja3": "9e1e7f3b...",
  "ja4": "t12i050500_ab12cd34ef56_98fe76dc54ba",
  "alpn": null
}
```

6.3 Where the data comes from during the hackathon

Need	Tool / source	Output
Benign background load	iperf3, Ostinato, TRex, ordinary browsing	pcap
SYN / UDP floods	hping3	pcap
Port scans and sweeps	nmap	pcap
Slow HTTP exhaustion	Slowloris	pcap
DNS tunnelling	iodine, dnscat2	pcap
DGA names	Published DGA algorithms, DGArchive (on request)	Name lists to query
C2 beaconing	Sandboxed C2 emulator or a simple jittered check-in script	pcap

Need	Tool / source	Output
Labelled public data	CIC-IDS2017/2018, CIC-DDoS2019, CTU-13, UNSW-NB15, CIC-Bell-DNS-EXF-2021	pcap + labelled CSV
Flow records	softflowd -r capture.pcap -n 127.0.0.1:2055	NetFlow v5/v9 over UDP
Everything at once, deterministically	Our own enclave synth generator	A single labelled demo pcap plus matching intel files

6.4 What we will never receive

- Decrypted content or TLS session keys.
- A way to request missing data or retransmissions.
- Labels on live traffic - only training data is labelled.
- Endpoint or host telemetry (process names, file hashes).
- Guaranteed completeness: sFlow is sampled, NetFlow may be unidirectional, mirrors drop under load.

7. Encryption: what is hidden and what stays visible

```
[ Ethernet ][ IP header ][ TCP/UDP header ][ TLS record hdr ][ ##### encrypted payload ##### ]
  visible   visible     visible             visible (type, len)                NEVER TOUCHED
```

Protocol	Hidden	Visible and useful
TLS 1.2	Application data	IPs, ports, sizes, timing, ClientHello (-> JA3/JA4), SNI, cipher list, certificate
TLS 1.3	Application data and the certificate	IPs, ports, sizes, timing, ClientHello/ServerHello, SNI (usually), supported versions
QUIC / HTTP3	Almost everything after the Initial packet	UDP 443 flow shape, version, connection-ID lengths, packet sizes and timing
DNS (UDP 53)	Nothing - it is cleartext	Full query name, record type, response code (NXDOMAIN), answer size
DoH / DoT	Query names	Only flow behaviour (sizes, timing, destination)
IPsec / WireGuard	Inner packets and inner addresses	Outer IPs, sizes, timing
SSH	Commands and output	IPs, ports, version banner, sizes, timing

7.1 Two questions you will be asked

'If it is encrypted, where is the flaw you exploit?'

Encryption protects content from everyone, including defenders - and attackers rely on that. But encryption does not hide behaviour: who talks to whom, how often, in what sizes, in what rhythm, and how much. A DDoS needs no content at all; a scan is just knocking on doors; a beacon is a clock. The envelope analogy: you cannot open sealed envelopes, but you can see the sender, the recipient, the size, the frequency, and that 10,000 arrived in one minute.

'Why is QUIC different?'

QUIC Initial packets are protected with keys derived from the connection ID, which are public, so many tools unprotect them to read the ClientHello. We deliberately do not: removing that protection is decryption, and constraint (b) forbids it. We use QUIC's cleartext long-header fields, packet sizes and timing instead. Saying this out loud shows the judges you read the constraint carefully.

8. Our solution: the architecture end to end

```

replay / live mirror / NetFlow          (simulated diode: one-way UDP, enclave has no route out)
    |
    v
[1 INGEST] pcap/pcapng reader | NetFlow v5 collector | (IPFIX/sFlow via goflow2 - roadmap)
    |               read-only, SHA-256 of every capture recorded as the evidence source
    v
[2 FLOW METER + METADATA] bidirectional flows (active 5 s / idle 15 s / unanswered 2 s timeouts)
    |               DNS question/answer events, TLS ClientHello -> JA3 + JA4, SPLT
    v
[3 BOUNDED BUS] asyncio queue: files apply backpressure, live feeds drop-and-count
    v
[4 DETECTORS] ddos-stat | beacon-score | dns-lexical (DGA + tunnel) | tls-meta | scan-trw | exfil-baseline
    |               each emits Detection{class, confidence, evidence, top factors, MITRE, summary, action}
    v
[5 FUSION] allowlist -> asset-weighted severity -> de-duplication -> campaign correlation
    v
[6 SINKS] hash-chained JSONL evidence log | in-memory store | WebSocket push | REST API
    v
[7 DASHBOARD] live feed, KPIs (flows/s, p95 latency, drops), evidence drawer, schema, guardrails

```

8.1 Design principles

Principle	What it means in code
Passive by construction	Sources only listen or read; an egress guard makes outbound connects fail loudly
One format inside	Every input becomes FlowRecord / DnsEvent / TlsEvent, so detectors never care where data came from
Event time, not wall time	Windows close on traffic timestamps, so a 20x replay produces the same alerts as real time
Bounded memory	Every per-entity table is an LRU with a hard cap; spoofed floods cannot exhaust RAM
Explain everything	Every detection carries feature values, thresholds, weighted contributions and a plain summary
No magic numbers	Every threshold lives in core/constants.py with the reason or source beside it
Degrade, don't fabricate	Missing inputs switch detectors off with a stated reason instead of guessing
Deterministic	Same input -> same alerts; seeded generators; offline models with recorded hashes

8.2 Module map (layering)

```

cli / api          -> parse input, call the pipeline, return results
pipeline          -> source -> bus -> detectors -> fusion -> store
detectors/*       -> business logic per threat class (no I/O)
features/*        -> shared feature code (windows, lexical DGA features)
fusion/*          -> severity, de-duplication, correlation
ingest/*          -> read-only sources, flow meter, DNS/TLS parsers, Community ID
sinks/*           -> alert store, hash-chained log
core/*            -> constants, config, logging, exceptions, statistics, scoring (imports nothing above)

```

9. The detectors in depth

Every detector follows the same pattern: maintain small per-entity state, compute named features, score each feature against its threshold with a logistic curve (0.5 exactly at the threshold), combine with fixed weights into a confidence, and raise a detection only above 0.5. The weighted contributions become the alert's 'top factors'. ML models (section 10) plug into the same interface and replace or refine the statistical score.

```
strength(feature) = 1 / (1 + exp(-k * (value/threshold - 1)))    k = 4  (lower-is-worse features invert the ratio)
confidence        = sum_i( w_i * strength_i ) / sum_i( w_i )
top factors       = sorted( w_i * strength_i / sum(w) )
```

9.1 DDoS (ddos-stat)

Aspect	Detail
Entity / window	Target IP; tumbling 5-second event-time windows
Features	New flows/s; z-score vs EWMA baseline of that target; source-IP Shannon entropy; share of TCP flows that never completed a handshake; unsolicited bytes on amplification ports (19, 53, 123, 161, 389, 1900, 11211) and response/request byte ratio; TTL spread
Thresholds	Floor 500 new flows/s; z >= 6; entropy >= 10 bits (~1,000 evenly spread sources); SYN-only >= 0.8; amplification >= 10x and >= 10 Mbit/s unsolicited
Sub-types	Spoofed SYN flood, SYN flood, UDP reflection/amplification, volumetric flood
Why entropy works	Legit busy servers see a few thousand repeat clients (5-8 bits); forged random sources push entropy toward log2(number of sources), e.g. 118,000 sources -> 16.8 bits
False positives	Flash crowds (sale day, results day) raise rate but keep handshakes complete and entropy moderate; baseline poisoning is prevented by not updating during alerts
Evasion	Low-rate application-layer floods (Slowloris) look different: many long-lived, nearly idle connections - add a 'concurrent half-idle connections per target' feature
MITRE	T1498.001 Direct Network Flood, T1498.002 Reflection Amplification

9.2 C2 beaconing (beacon-score)

Aspect	Detail
Entity	(internal source, destination, port, protocol) pair; last 64 connection starts
Features	Mean inter-arrival; coefficient of variation (CV = std/mean) of inter-arrivals; CV of bytes per session; number of repetitions; how many internal hosts call the same destination
Thresholds	>= 6 connections; mean interval >= 5 s; interval CV <= 0.2 (tolerates about 20% jitter); bytes CV <= 0.5; a destination called by 3+ hosts is treated as a shared service
Worked example	Gaps of 60 s +/- 1.3 s -> CV 0.021 (human browsing is > 0.6) -> interval strength ~1.0
False positives	NTP, update checks, monitoring agents, package mirrors. Handled by destination prevalence, the allowlist and analyst feedback
Evasion	Heavy random jitter and sleep schedules. Roadmap: Lomb-Scargle periodogram and inter-arrival histogram modality, which survive jitter far better than CV
MITRE	T1071.001 Web Protocols, T1573 Encrypted Channel

9.3 DGA and DNS tunnelling (dns-lexical)

Aspect	DGA	Tunnelling
Entity / window	Client host; 60 s	(client, registered domain); 300 s

Aspect	DGA	Tunnelling
Features	Per name: character entropy, share of letter pairs outside the common-English bigram set, digit ratio, length. Per host: count of DGA-like names, NXDOMAIN ratio, mean name score	Unique subdomains, mean longest label length, TXT/NULL record share, query count
Thresholds	Name score ≥ 0.6 ; ≥ 20 such names per window; NXDOMAIN ratio ≥ 0.5	≥ 100 unique subdomains; mean label ≥ 40 chars (base32 chunks are 40-63); TXT/NULL share ≥ 0.5
Trigger	Fires as soon as the count reaches the threshold, again at each doubling - no waiting for the window to end	Same doubling trigger
False positives	CDN and telemetry names (long hashes) - they resolve (NOERROR), so the NXDOMAIN signal stays low	Anti-virus and CDN lookups with encoded labels; handled by TXT share and allowlist
Evasion	Dictionary DGAs (word-based) defeat entropy - the character CNN model (section 10) handles them	Low-and-slow tunnels under the unique threshold; roadmap: per-domain bytes-per-day budget
MITRE	T1568.002 Domain Generation Algorithms	T1071.004 DNS, T1572 Protocol Tunnelling

9.4 Malware in encrypted sessions (tls-meta)

Aspect	Detail
Input	TLS ClientHello parsed in cleartext; nothing after the handshake is read
Fingerprints	JA3 (MD5 of version, ciphers, extensions, groups, point formats, GREASE removed) and JA4 (t/q, version, SNI flag, cipher and extension counts, ALPN, truncated SHA-256 of sorted ciphers and of sorted extensions + signature algorithms)
Features	Offline blocklist match; local rarity (how many internal hosts already use this JA4); SNI absent; roadmap: SPLT sequence and server fingerprint JA3S/JA4S, certificate traits for TLS 1.2
Warm-up	Rarity only counts after 200 handshakes have been seen, so a fresh sensor does not flag everything
Rule	Blocklisted -> always raised (confidence from the combined signals); otherwise raised only above 0.6
Evasion	uTLS-style fingerprint mimicry copies Chrome's ClientHello. Counter: SPLT rhythm and server-side fingerprints, which the malware cannot easily fake
MITRE	T1573.002 Asymmetric Cryptography channel, T1071.001

9.5 Reconnaissance (scan-trw)

Aspect	Detail
Entity	Source IP; state resets after 60 s of silence
Method	Threshold Random Walk on first-contact outcomes: success adds $\ln(0.2/0.8) = -1.39$, failure adds $\ln(0.8/0.2) = +1.39$; declare scanner at $\ln(0.99/0.01) = 4.6$ (about four net failures)
Extra gates	≥ 20 distinct targets; failed ratio ≥ 0.5
Shape	Vertical (ports $\geq 5 \times$ hosts), horizontal (hosts $\geq 5 \times$ ports), block (both)
OT awareness	Port 502 (Modbus) sweeps add MITRE ICS T0846 Remote System Discovery
False positives	Vulnerability scanners and inventory tools - allowlist them by address
Evasion	Very slow scans across hours: TRW state resets after 60 s; roadmap: long-window per-source distinct-target sketch (HyperLogLog)

9.6 Exfiltration (exfil-baseline)

Aspect	Detail
Entity / window	Internal host; 60 s windows plus an EWMA baseline of bytes out per window
Direction	Needs the internal address ranges (a user input). Works on bidirectional flows and on one-way NetFlow records because it aggregates per host
Features	Bytes out, out/in ratio, producer-consumer ratio, upload z-score vs the host's own history, top destination and whether it was ever contacted before
Thresholds	≥ 50 MiB out in a window; PCR ≥ 0.8 ; $z \geq 4$ once the baseline has 5 windows
False positives	Backups, video uploads, software builds - allowlist their destinations; baselines learn regular jobs
Evasion	Slow drip exfiltration under the per-window threshold; roadmap: daily cumulative budget per host-destination pair
MITRE	T1041 Exfiltration over C2 channel, T1048 Exfiltration over alternative protocol

9.7 Campaign correlation (the extra mile)

Alerts on the same internal host within 15 minutes are linked into a single campaign alert: e.g. DGA lookups -> C2 beaconing -> exfiltration. Confidence grows with each stage (0.90 for two stages, +0.03 per extra stage). This turns three separate tickets into one incident story, which is what an analyst actually needs, and it is a feature most open tools do not provide.

10. How the AI/ML is used

10.1 No API keys, no cloud AI

All models are trained by the team and run inside the enclave. There is no API key anywhere in the system. Cloud AI services are ruled out for six reasons:

Risk	Why it matters
Breaks the architecture	The enclave would need an internet path out - exactly what the diode design exists to prevent
Metadata is sensitive	IPs, hostnames, domains and who-talks-to-whom form a map of national critical infrastructure
Third-party retention	The provider may log requests; the data leaves national control
Key theft	Keys in code or config leak and get abused
Availability	Rate limits or outages would stop detection
Non-determinism	Model versions change silently, so the same evidence could be judged differently - bad for forensics

If you want generative AI in the story: run a small open-weights language model locally inside the enclave (for example via Ollama) only to phrase alert summaries in plain English. It never decides anything and the system works with it switched off. During development, AI coding assistants are fine - but never paste real captured traffic into any cloud tool.

10.2 Model portfolio

Detector	Statistical layer (runs day one)	ML layer	Why this model
DDoS	EWMA baseline, robust z-score, entropy, CUSUM	LightGBM sub-type classifier on window features	Fast, tabular, explainable with TreeSHAP; handles skewed features
Beaconing	Interval CV, repetition, prevalence	LightGBM on interval/size statistics and periodogram power	Small feature vectors; robust to scale
DGA	Entropy, rare bigrams, NXDOMAIN rate	LightGBM on lexical features + a small character-level CNN (exported to ONNX)	The CNN learns dictionary-style DGAs that entropy misses
DNS tunnel	Label length, uniqueness, TXT share	Isolation Forest on per-domain aggregates	Unsupervised: tunnels are rare and varied
Encrypted malware	Blocklist, rarity, SNI	LightGBM on flow statistics + SPLT (first 20 sizes and gaps)	Proven in encrypted-traffic literature; no payload needed
Scan	Threshold Random Walk	Not needed - TRW is optimal for the question it answers	Keep it simple and exact
Exfiltration	Per-host EWMA, PCR	Isolation Forest on host-window features	Unsupervised; every host has its own normal

10.3 Training pipeline

1. Collect: public labelled datasets plus our own lab captures (section 17).
2. Extract features with the **same code** the live pipeline uses (features/*), so training and inference never drift.
3. Remove identity features (IP addresses, exact ports, timestamps) so the model learns behaviour, not addresses.
4. Split **by time** (train on earlier days, test on later days) - random splits leak and inflate scores.
5. Train LightGBM / Isolation Forest / CNN with seeds fixed in constants.
6. Calibrate probabilities with isotonic regression on a held-out validation set.
7. Evaluate (section 10.4); write a model card; export model file + SHA-256 into the offline bundle.
8. Import into the enclave through the approved one-way path; the loader refuses any file whose hash does not match.

10.4 Validation metrics that matter

Metric	Why
Per-class precision, recall, F1, PR-AUC	Accuracy is meaningless when attacks are 0.1% of traffic
False alarms per hour on benign-only replay	What an operator actually feels
Cross-dataset test (public -> our lab)	Proves generalisation beyond one dataset's artefacts
Expected calibration error + reliability plot	Proves the confidence numbers mean what they say
Time-to-detect per class	How long an attack runs before it is visible
Processing latency p50/p95	Pipeline speed, separate from time-to-detect
Throughput sustained without drops	Constraint (d)

Known dataset traps to mention: CIC-IDS2017 has documented labelling and flow-meter errors (use the corrected release by Engelen et al., 2021); many papers report 99% accuracy only because of random splits and IP-address leakage. Saying you avoided these earns credibility instantly.

10.5 Feedback loop

Analysts mark alerts as confirmed or false positive on the dashboard. Labels are stored inside the enclave and used for the next offline retraining cycle. Nothing leaves the enclave automatically; if models are retrained elsewhere, labels leave only through an approved, audited procedure.

11. The alert record

Every alert is a Pydantic model, exported as JSON Schema (GET /api/schema/alert). Required PS fields are all present: timestamp, flow identifier, threat class, confidence and evidence.

```
{
  "schema_version": "1.0",
  "alert_id": "enclave-sensor-01-00000004",
  "detected_at": "2026-09-17T10:03:25Z", "event_start": "...", "event_end": "...",
  "flow_id": "1:LQU9qZlK+B5F3KDmev6m5PMibrg=", "src": "118003 sources", "dst": "10.10.0.5", "dst_port": "443", "proto": "TCP",
  "sensor_id": "enclave-sensor-01", "input_mode": "pcap", "host": "10.10.0.5",
  "threat_class": "ddos", "sub_type": "Spoofed SYN flood",
  "confidence": 0.97, "severity": "critical", "severity_score": 97,
  "asset": {"address": "10.10.0.5", "name": "Public web gateway", "zone": "DMZ", "criticality": 4},
  "model": {"name": "ddos-stat", "version": "1.0"},
  "mitre_attack": ["T1498.001"],
  "summary": "Spoofed SYN flood toward 10.10.0.5: 3,000 new flows/s from 18,000 sources ...",
  "suggested_action": "Ask the upstream provider for scrubbing or rate-limiting toward 10.10.0.5.",
  "evidence": {"new_flows_per_s": {"observed": 3000.0, "reference": "baseline 2.1/s"},
    "src_ip_entropy_bits": {"observed": 14.1, "reference": "alert at 10.0"}, ...},
  "top_factors": [{"feature": "flow_rate_z", "contribution": 0.35}, ...],
  "events_merged": 7, "related_alert_ids": [], "processing_latency_ms": 0.42,
  "custody": {"source_ref": "pcap:demo.pcap:sha256:...", "prev_alert_hash": "...", "alert_hash": "..."}
}
```

11.1 Severity

```
severity_score = 100 x confidence x class_impact x (0.6 + 0.1 x asset_criticality) capped at 100
class_impact: ddos 1.0, exfiltration 1.0, campaign 1.0, encrypted_malware 0.95, c2_beacon 0.9,
              dns_tunnel 0.9, dga 0.85, recon_scan 0.7
buckets:      >= 80 critical | >= 60 high | >= 40 medium | else low
```

11.2 De-duplication

A flood produces one alert that updates, not thousands. Detections with the same (class, entity) within 5 minutes merge into the open alert: the event count and end time grow, and evidence is refreshed when the new detection is at least as confident.

11.3 Chain of custody

Each new alert (and each update) is appended to a JSONL log where every line stores the previous line's hash and SHA-256(previous hash + canonical record). Changing any past line breaks every hash after it; enclave verify-log recomputes the chain. The capture file's SHA-256 is recorded as the evidence source. This is a lightweight, blockchain-style ledger - it fits the 'Blockchain & Cybersecurity' theme without needing a distributed network.

12. Streaming, latency and throughput

12.1 How streaming is achieved

- The source yields events one by one; nothing waits for the end of a file.
- Long flows are exported every 5 s (active timeout); silent flows close after 15 s; flows that never got a reply close after 2 s, so floods and scans surface fast.
- Detectors keep event-time windows; the pipeline flushes them each second of event time.
- Count-based detectors (scan, DGA, tunnel, exfil) fire as soon as evidence is sufficient via a doubling trigger.
- A bounded queue separates ingest from analysis: file replays apply backpressure; live feeds drop and count, mirroring what a real diode would do.

12.2 Two latencies - never confuse them

Measure	Definition	Target
Processing latency	Time from an event entering the bus to the alert being emitted	p95 $\leq$ 5 s (in practice milliseconds)
Time-to-detect	Time from attack start to the first alert - bounded by timeouts, windows and how much evidence the class needs	Flood / scan: seconds. Beacon: 6 repetitions by definition. Exfil: as soon as 50 MiB has left

12.3 Throughput target and how to prove it

Item	Plan
Stated target	20,000 flows/s sustained, 4-core laptop, single process; stretch 50,000 with partitioned workers
Method	Replay a fixed pcap / NetFlow stream at 1k, 10k, 20k, 50k flows/s for 10 minutes each
Record	Flows processed per second, p50/p95 processing latency, queue depth, dropped records, CPU and RAM
Report	A results table in the docs and on the dashboard - only measured numbers, never estimates
Scale-out path	Partition by hash of entity (target IP for DDoS, source IP for scan...) across worker processes; a Kafka/Redpanda bus between sensor and workers

13. The dashboard and the demo

13.1 What the dashboard shows

Area	Content	PS requirement it proves
One-way badge	Blocked outbound attempts (always 0 in normal operation)	Read-only ingest
Pipeline strip	Source -> sensor -> bus -> detectors -> store, with status	Architecture
KPI tiles	Flows/s vs target, p95 latency vs budget, open critical alerts, detectors running, dropped records	Streaming and throughput
Live detections	Time, severity chip, class, source -> target, confidence, status	Labelled alerts with confidence
Evidence drawer	Summary, evidence table, top factors, suggested action, MITRE, custody hashes, raw JSON	Supporting evidence, schema
Detectors panel	Which detectors run for this input and why others do not	Graceful degradation

13.2 What the user inputs

Input	Why
Traffic source: capture file, live mirror, NetFlow collector port, replay speed	Chooses what is analysed
Internal address ranges	Separates outbound from inbound (needed for exfiltration)
Asset inventory with criticality 1-5	An attack on a SCADA HMI outranks one on a printer
Allowlist	Silences known-good behaviour (NTP, backup targets, scanners)
Sensitivity per detector	Trades missed attacks against false alarms
Offline intelligence and model bundles	Updates without internet, verified by hash
Analyst actions: acknowledge, confirm, false positive, notes	Human decision; labels feed retraining

What the user never inputs: firewall credentials, production access, or API keys.

13.3 Demo script (5 minutes)

1. Open with the architecture strip: 'traffic flows one way; the badge proves nothing leaves.'
2. Start the replay of the synthetic scenario at 20x. Scan on the SCADA HMI appears first.
3. DGA lookups from a finance workstation, then the spoofed SYN flood: open it and read the entropy, SYN-only ratio and top factors.
4. Amplification flood on the DNS server: 'few flows, huge bytes - a different signature, same detector.'
5. Blocklisted TLS client: 'fingerprint only; nothing decrypted.'
6. Beaconing, then exfiltration from the same finance PC: the campaign alert links DGA -> beacon -> exfiltration.
7. Show the raw JSON alert and the hash chain; run `enclave verify-log live`.
8. Switch to a NetFlow-only source: 4 of 6 detectors run, the other two say why.
9. Close on the performance table: stated target, measured numbers, dropped records.

14. The prototype we built

The repository is a Python 3.12 package called `enclave` with a small built-in web dashboard. It runs on Windows, Linux or macOS with no internet access once dependencies are installed.

14.1 Repository layout

```
Unidirectional IP Traffic/
pyproject.toml          dependencies, ruff, mypy --strict, pytest config
README.md              quick start, commands, known limitations
config/enclave.example.json internal ranges, asset inventory, allowlist
intel/                  offline blocklists + manifest.json (SHA-256 per file)
src/enclave/
  core/                 constants.py (every threshold, with reasons) | config.py | exceptions.py
                        logging.py (JSON logs + run id) | stats.py (entropy, CV, EWMA, TRW, PCR)
                        scoring.py (logistic signal combination) | lru.py (bounded state)
  schema/               events.py (FlowRecord, DnsEvent, TlsEvent) | alert.py (Alert JSON schema)
  ingest/               pcap_source.py | netflow_source.py | flowmeter.py | community_id.py
                        parsers/dns.py | parsers/tls.py (JA3 + JA4)
  features/             windows.py (event-time windows, doubling trigger) | lexical.py (DGA features)
  detectors/            ddos.py beacon.py dns.py tls.py scan.py exfil.py | registry.py | base.py
  fusion/               engine.py (severity, de-dup) | correlator.py (campaigns)
  sinks/                store.py (alert store, WebSocket fan-out, hash-chained log, verifier)
  intel.py              offline intelligence loader with hash verification
  egress_guard.py        process-level block on outbound connections
  metrics.py            throughput, latency reservoir, coverage
  pipeline.py           source -> bus -> detectors -> fusion -> sinks
  api/                  FastAPI app (REST + WebSocket) and the static dashboard
  synth.py              deterministic synthetic scenario generator (demo capture + intel)
  cli.py                enclave synth | replay | netflow | verify-log | intel-manifest
  tests/                mirrors src/enclave; unit tests per module + end-to-end scenario test
```

14.2 How to run it

```
# one-time setup (on a machine with internet)
python -m venv .venv
.venv\Scripts\activate           (Linux/macOS: source .venv/bin/activate)
pip install -e ".[dev]"

# 1. generate the labelled synthetic demo capture + matching offline intel
enclave synth --out data/demo.pcap --intel intel

# 2. replay it at 20x and open the dashboard at http://127.0.0.1:8000
enclave replay data/demo.pcap --speed 20 --config config/enclave.example.json --serve

# 3. replay as fast as possible and print a summary (throughput check)
enclave replay data/demo.pcap --speed 0 --config config/enclave.example.json

# 4. flow-only mode: listen for NetFlow v5 records
enclave netflow --listen 127.0.0.1:2055 --config config/enclave.example.json --serve

# 5. verify the evidence chain
enclave verify-log var/alerts.jsonl

# 6. tests and static checks
pytest -q
ruff check src tests
mypy
```

14.3 What is built and what is roadmap

Component	Status	Notes
pcap/pcapng replay with speed control	Built	Ethernet, raw IP, Linux SLL; IPv4 and IPv6
NetFlow v5 collector	Built	Listen-only UDP; unidirectional records handled

Component	Status	Notes
IPFIX / NetFlow v9 / sFlow	Roadmap	Template decoding, or a goflow2 front end whose JSON we consume
Flow meter with active / idle / unanswered timeouts	Built	Bounded flow table; SPLT captured
DNS parsing, TLS ClientHello, JA3, JA4	Built	GREASE-aware; QUIC left sealed by design
Six detectors + campaign correlation	Built (statistical layer)	ML layer plugs into the same interface
LightGBM / Isolation Forest / char-CNN models	Roadmap	Training scripts, model cards, hash-verified bundle
Severity, de-dup, allowlist, asset weighting	Built	
Hash-chained evidence log + verifier	Built	
REST + WebSocket API, JSON Schema endpoint	Built	
Live dashboard	Built (lean)	Richer UI shown in the interactive mock-up
Egress guard	Built	Blocks outbound connect/sendto at process level
Benchmark harness	Roadmap	Fixed-rate replays with a results table
Docker Compose with simulated diode and internal-only network	Roadmap	Shows the physical one-way boundary

14.4 Testing strategy

- Unit tests per module, mirroring the source tree.
- Maths edge cases: empty inputs, zero means, perfectly regular vs jittered intervals, entropy of one vs many sources.
- Parsers tested against constructed ClientHello and DNS messages, including GREASE values and truncation.
- End-to-end: generate the synthetic scenario, replay it, and assert every threat class is raised, the campaign links the right stages, and the hash chain verifies.
- Egress test: an outbound connection attempt under the guard raises the typed error.
- No test makes a network call.

14.5 The synthetic scenario

The generator writes one deterministic, seeded capture of about ten minutes: benign background (HTTPS sessions with realistic ClientHellos, DNS lookups, periodic NTP) plus one labelled segment per threat class. It fabricates packets directly into a file - it never sends anything on a network - so it is safe to run anywhere and gives every team member the same reproducible demo. Large uploads are stored header-only (like a snap-length capture), keeping the file small while byte counters stay realistic.

15. Data for training and evaluation

15.1 Public datasets

Dataset	Use it for	Caveat
CIC-IDS2017 (corrected release)	DDoS, scans, botnet, brute force	Use the Engelen et al. (2021) corrections; avoid random splits
CSE-CIC-IDS2018	Larger multi-day attack mix	Heavy; sample by day
CIC-DDoS2019	Reflection/amplification and SYN floods	Very imbalanced; balance per window, not per packet
CTU-13 (Stratosphere)	Real botnet C2 traffic with background	Labels at host/flow level; good for beaconing
UNSW-NB15	Mixed modern attacks	Different feature extractor; retrain features from pcaps
CIC-Bell-DNS-EXF-2021	DNS exfiltration and tunnelling	Mostly DNS; pair with benign DNS
CIRA-CIC-DoHBrw-2020	DNS over HTTPS tunnels	Flow-only DoH detection
DGArchive / open DGA implementations	DGA name families	DGArchive needs an access request
Tranco top-1M list	Benign domain names for DGA negatives	Popular names are not automatically safe
abuse.ch SSLBL	Known-bad JA3 fingerprints	Import offline with hash manifest

15.2 Lab-generated traffic

The PS lists iperf3 / Ostinato / TRex for benign load and hping3, Slowloris, iodine / dnscat2 and a sandboxed C2 emulator for attack traffic. Rules for using them responsibly:

- Only inside an isolated virtual lab network you own - never on college, office, cloud or public networks.
- Record captures on the lab's own interfaces; keep the lab disconnected from the internet.
- Get your mentor's or institution's written approval before generating any attack traffic.
- For most of the build and the demo, the built-in synthetic generator is enough and carries no risk.

15.3 Labelling

Label by time interval and host: keep a simple log of when each lab scenario ran and from which VM. A flow is labelled with the scenario whose interval and host match; everything else is benign. Store labels beside the capture with its SHA-256 so results are reproducible.

16. Novelty - what is genuinely new

Most individual techniques are known (entropy, JA3/JA4, TRW, beacon scoring). The novelty is the system: a pipeline designed around the one-way constraint from the first line of code. Claim it honestly - judges respect that more than inflated claims.

#	Novelty point	Existing tools	Ours
1	Passive by design and by test	Built for two-way networks; active features must be switched off	Every feature is computable from one-way data; an egress guard and a test prove it; QUIC left sealed
2	One pipeline, six classes	Four or five separate tools stitched together	Shared flow/metadata layer, one alert format, one dashboard
3	Hybrid statistics + ML	Pure thresholds (noisy) or pure ML (cold start)	Statistics work on day one; ML refines; same explainable interface
4	Calibrated confidence	Unitless anomaly scores	Confidence that means a probability; asset-weighted severity
5	Explanation in every alert	Black-box 'anomaly detected'	Feature values vs thresholds, weighted top factors, summary and suggested action
6	Graceful degradation	Assume one input format	Packets -> 6 detectors; flow records -> 4, with the reason shown
7	Tamper-evident evidence	Plain editable logs	Hash-chained alert ledger plus capture SHA-256
8	Honest measurement	Accuracy only, often leaked	Throughput, two latencies, false alarms per hour, cross-dataset tests
9	Campaign correlation	One ticket per stage	Stages on one host become one incident story
10	Air-gap native	Cloud lookups and updates	Offline intelligence and models with hash verification; no API keys

Answer to 'Isn't this just Zeek + RITA?': Zeek produces logs and RITA scores beacons from imported logs. We turn a one-way stream into real-time, scored, explained alerts across all six classes - from packets or flow records alone - with guaranteed passivity, calibrated confidence, a tamper-evident ledger and published performance numbers.

17. Limitations, risks and mitigations

Limitation	Impact	Mitigation
Payload-level attacks inside HTTPS (e.g. injection)	Invisible to metadata	Out of scope; production-side WAF and endpoint security
Encrypted DNS (DoH/DoT)	No query names for DGA/tunnel detection	Flow-based DoH tunnel model; policy to force internal resolvers
TLS 1.3 hides certificates; fingerprint mimicry	Weaker TLS signals	SPLT rhythm, server fingerprints, rarity
Slow and low attackers	Stay under thresholds	Long-window baselines, daily budgets, campaign correlation
Legitimate periodic or bulk traffic	False positives	Allowlist, destination prevalence, calibrated confidence, analyst feedback
No confirmation or blocking possible	Response depends on people	Clear evidence and a suggested manual action for the production team
Blind spots: unmirrored links, VPN inner traffic, NAT	Missed activity	TAP placement review; document coverage
Capture or diode loss	Gaps	Count and display drops; never claim full coverage
Concept drift and adversarial ML	Degrading models	Periodic offline retraining; statistical layer as a floor
Lab data differs from real networks	Optimistic metrics	Cross-dataset tests; local baselines
Python single-process throughput	Ceiling on one core	Partitioned workers; critical paths in compiled code later

Never the only defence. This system is the eyes. Firewalls, endpoint protection, patching and access control on the production side are the locks and guards. Together that is defence in depth.

18. Deployment and scaling

18.1 Reference deployment

Machine	Role	Indicative sizing (to be validated by benchmark)
Sensor	Capture, flow metering, DNS/TLS parsing	16+ cores, 64 GB RAM, capture NIC matching the link speed
Analytics	Bus, detectors, models, alert store	16 cores, 64 GB RAM, NVMe; no GPU needed
Storage	Rolling capture buffer, flow archive, evidence log	NVMe for hot data; retention per policy
Dashboard	API and web UI	Can share the analytics server
Analyst workstations	Browser only	Enclave LAN, no internet

18.2 Scaling path

1. Single process (prototype): measure the ceiling honestly.
2. Multiple worker processes partitioned by entity hash (target IP for DDoS, source IP for scans).
3. A Kafka-compatible bus (e.g. Redpanda) between sensors and workers; multiple sensors behind a packet broker.
4. Move the flow meter to a compiled sensor (Zeek, Suricata or nProbe) at 10G+ and consume its metadata.

18.3 Hackathon deployment

One laptop. Optionally Docker Compose with a replay container, a 'diode' container that forwards UDP one way, and enclave containers on a network marked internal (no route out).

19. Compliance, ethics and evidence handling

- **Indian context.** NCIIPC (under NTRO, created by Section 70A of the IT Act, 2000) protects critical information infrastructure; CERT-In directions (April 2022) require reporting of specified incidents within 6 hours - clear, evidence-backed alerts shorten that path.
- **Personal data.** IP addresses and domain names can be personal data under the Digital Personal Data Protection Act, 2023. Minimise: store metadata only, set retention periods, restrict access.
- **Chain of custody.** Hash every capture on arrival; keep captures read-only; hash-chain alerts; log who viewed or exported evidence.
- **Industrial security.** IEC 62443 zone-and-conduit thinking matches the diode model; place sensors per zone.
- **Responsible testing.** Attack traffic only in an owned, isolated lab with approval.

20. Team plan

20.1 Roles for a six-person team

Role	Owns
Lead / architect	Architecture, constraints compliance, integration, the pitch
Ingest engineer	Flow meter, parsers, NetFlow/IPFIX, replay and benchmarks
Detection engineer 1	DDoS, scan, exfiltration detectors and their tests
Detection engineer 2 / ML	DGA, tunnel, TLS, beacon; model training, calibration, model cards
Frontend engineer	Dashboard, WebSocket feed, evidence drawer, demo flow
Data and documentation	Datasets, labelling, validation report, README, slides, Q&A rehearsal

20.2 36-hour hackathon timeline

Hours	Goal	Exit criterion
0-3	Freeze architecture and alert schema; set up repo, CI, synthetic scenario	Everyone can replay the demo capture
3-10	Flow meter, DNS/TLS parsing, DDoS + scan + exfil detectors	Three classes alert end to end
10-18	DGA, tunnel, TLS, beacon; fusion; hash-chained log	All six classes plus campaign
18-24	Dashboard live feed and evidence drawer; API; egress guard	Demo runs from a clean clone
24-30	Benchmarks, validation numbers, one ML model trained and calibrated	Results table filled with measured numbers
30-34	Slides, documentation, rehearsal x3, Q&A drill	Pitch fits 5 minutes
34-36	Buffer: fix, freeze, back up the demo (screen recording)	Tagged release

21. Judge question bank

Question	Strong answer
How is this different from Shorot/Suricata?	They match payload signatures; we detect behaviour from metadata, which works on encrypted traffic and needs no return path.
You cannot decrypt - how do you find malware in TLS?	Handshake fingerprints (JA3/JA4), local rarity, missing SNI, blocklists, and the size/timing rhythm of the first packets.
Why not decrypt QUIC Initial packets? The keys are public.	Removing that protection is decryption; the PS forbids it. We use cleartext header fields, sizes and timing.
How do you prove read-only?	Listen-only sources, an egress guard that raises on any outbound connect, a test that proves it, and zero blocked attempts on the dashboard.
What throughput did you test?	State the measured number from the benchmark table, the hardware and the drop count - never an estimate.
What is your latency?	Two numbers: processing latency p95 (milliseconds) and time-to-detect per class (bounded by timeouts and evidence needed).
Accuracy?	Per-class precision/recall/PR-AUC, false alarms per hour on benign replay, cross-dataset results; accuracy alone is misleading with rare attacks.
How do you avoid false positives from NTP or updates?	Destination prevalence, allowlist, calibrated confidence and analyst feedback.
What if the attacker adds jitter to beacons?	CV tolerates about 20%; periodogram and interval-histogram features are the next layer.
What about dictionary DGAs?	Entropy misses them; the character-level CNN is trained for that case.
What if the mirror drops packets?	We count drops, show them, and design detectors that work on partial data.
Why LightGBM and not deep learning everywhere?	Tabular features, fast CPU inference, TreeSHAP explanations; deep learning only where sequences matter (DGA names, SPLT).
How do models get updated without internet?	Offline bundle through an approved one-way transfer, verified by SHA-256 before loading.
Do you use ChatGPT/Gemini APIs?	No. That needs an outbound path and would leak infrastructure metadata. Everything runs locally.
Where does your software run physically?	On enclave servers after the diode's receive side; nothing on production, nothing on the diode.
Why a hash chain - is this blockchain?	A blockchain-style ledger: each alert commits to the previous hash, so tampering is evident. No distributed consensus is needed inside one enclave.
Can it block attacks?	No, by design. It produces evidence and a suggested manual action for the production team.
How does severity work?	Confidence x class impact x asset criticality, bucketed into four levels.
What is the Community ID?	A standard flow hash so the same flow can be found in Zeek, Suricata and our alerts.
What happens with NetFlow only?	Four detectors run; DNS and TLS detectors switch off and say why.
How does it scale to 100 Gbps?	Hardware sensors at line rate, partitioned workers by entity hash, a Kafka-compatible bus.
What data did you train on?	Named public datasets plus lab captures, with time-based splits and identity features removed.
What is the weakest part?	Payload-only attacks are invisible and slow-and-low exfiltration is hard; we document both and mitigate.
What about privacy?	Metadata only, retention limits, access control, no data leaves the enclave.
Why is your solution better for NTRO specifically?	Air-gap native, sovereign, open, explainable, evidence-preserving - aligned with how NCIIPC-grade sites are run.

22. Pitch

22.1 Five-minute script

1. **Hook (20 s).** 'Critical infrastructure watches its links through a one-way data diode. It keeps attackers from pivoting in - but it also blinds most security tools. We built the brain that works entirely behind the diode.'
2. **Problem (40 s).** No probing, no decryption, no blocking, huge volume, encrypted traffic.
3. **Solution (60 s).** Architecture strip: ingest -> flows and metadata -> six detectors -> fusion -> evidence ledger -> dashboard.
4. **Live demo (120 s).** Scan, DGA, spoofed SYN flood, TLS fingerprint, beacon, exfiltration, then the campaign alert; open one alert's evidence; verify the hash chain; switch to NetFlow-only.
5. **Proof (30 s).** Throughput and latency table, false alarms per hour, egress test.
6. **Why us (20 s).** Passive by design, explainable, air-gap native, honest measurement.
7. **Close (10 s).** 'Watch everything. Touch nothing. Explain every alert.'

22.2 Slide outline

#	Slide
1	Title, team, one-line promise
2	The one-way world: TAP -> diode -> enclave
3	Why existing tools fail behind a diode
4	Six threats, six passive signals
5	Architecture
6	Inside a detector: features, thresholds, top factors
7	Alert schema and hash-chained evidence
8	Live demo
9	Performance and validation (measured)
10	Novelty vs existing tools
11	Limits we know and how we handle them
12	Roadmap and deployment

23. Checklists

23.1 Before the demo

- Fresh clone installs and runs offline; demo capture regenerated with the fixed seed.
- Dashboard opens with data already flowing; browser zoom and fonts checked on the projector.
- A screen recording of the full demo as a fallback.
- Benchmark table filled with measured numbers and the hardware used.
- Egress test and hash-chain verification ready to run live.
- Every team member can answer the top ten questions in section 21.

23.2 Compliance with the PS (tick before submission)

Requirement	Where it is met
Ingest, feature extraction, model inference, alert output	ingest/, features/ + flowmeter, detectors/ (+ models), fusion/ + sinks/
Documentation of models, features, training/validation	This document, sections 9-10; model cards in the repo
Dashboard with severity and confidence	api/ static dashboard
Read-only ingest	Listen-only sources; egress guard; test
No payload decryption	TLS parser stops at ClientHello; QUIC sealed
Streaming with bounded latency	Event-time windows, timeouts, latency metrics
Stated and demonstrated throughput	Benchmark table
Standard alert schema	Alert JSON Schema endpoint; Community ID

24. References and further reading

- Jung, Paxson, Berger, Balakrishnan - Fast Portscan Detection Using Sequential Hypothesis Testing, IEEE S&P 2004 (Threshold Random Walk).
- Corelight - Community ID flow hashing specification (github.com/corelight/community-id-spec).
- Salesforce - JA3: a method for profiling SSL/TLS clients (2017).
- FoxIO - JA4+ network fingerprinting (2023); the JA4 TLS client fingerprint is BSD-3 licensed.
- Anderson and McGrew - Identifying Encrypted Malware Traffic with Contextual Flow Data, AISec 2016.
- Sharafaldin, Lashkari, Ghorbani - Toward Generating a New Intrusion Detection Dataset (CIC-IDS2017), 2018.
- Engelen, Rimmer, Joosen - Troubleshooting an Intrusion Detection Dataset: the CICIDS2017 Case Study, 2021.
- Garcia et al. - An empirical comparison of botnet detection methods (CTU-13), 2014.
- Moustafa and Slay - UNSW-NB15 dataset, 2015.
- Plohmann et al. - A Comprehensive Measurement Study of Domain Generating Malware (DGArchive), USENIX Security 2016.
- Woodbridge et al. - Predicting Domain Generation Algorithms with Long Short-Term Memory Networks, 2016.
- US-CERT TA14-017A - UDP-Based Amplification Attacks.
- RFC 3954 (NetFlow v9), RFC 7011 (IPFIX), RFC 8446 (TLS 1.3), RFC 9000 (QUIC), RFC 8701 (GREASE), RFC 5737 / RFC 6598 (documentation and shared address space).
- MITRE ATT&CK Enterprise and ICS matrices.
- Active Countermeasures - RITA (Real Intelligence Threat Analytics).
- CERT-In directions under Section 70B(6) of the IT Act, 28 April 2022; NCIIPC guidelines; Digital Personal Data Protection Act, 2023; IEC 62443.

Values such as throughput and accuracy must come from your own measured runs. Where this document gives targets, treat them as goals to verify, not results.